Google Cloud Platform + AI Fundamentals

Universidad Austral
Facultad de Ingeniería
Primer Cuatrimestre 2026

Profesores
Mora Villa Abrille (BVilla@austral.edu.ar)
Rodrigo Pazos (RPazos@austral.edu.ar)

¿Qué es un dato?

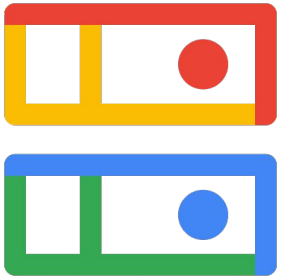
Un dato es una representación de información que puede ser almacenada, procesada y transmitida por un sistema.

Tipos de datos

- **Estructurados**: filas y columnas con un formato predefinido y fijo. Son fáciles de consultar y existen herramientas que hacen esas consultas muy eficientes. Son esenciales en procesos de negocio en prácticamente todas las industrias. (Ejemplos: SQL, Excel, CSV, etc.)
- **No estructurados**: datos que no siguen un modelo o esquema predefinido. Son más difíciles de consultar y presentan gran variedad de formatos. Generalmente requieren procesamiento adicional para extraer información útil. (Ejemplos: audio, video, imágenes, etc)
- **Semiestructurados**: datos que no siguen un esquema rígido pero incluyen etiquetas que describen su estructura. (JSON, XML)

Diferentes momentos requirieron diferentes tecnologías

- SQL en los 70 fue una solución fácil de usar y que se fue optimizando tanto para acceso, escritura y almacenamiento. Entre los 70 y los 90 el precio del GB pasó de 250K USD a 15 USD.
- En los 2000 la adopción de Internet llevó a una explosión de información y de usuarios y los motores de SQL dejaron de ser performantes. Aparecieron soluciones que escalan, en una categoría llamada NoSQL.
- Los 2010 con la data como lo más valioso que una empresa tiene ofrecen soluciones cada vez más sofisticadas para el universo de Big Data. Se popularizan los data lakes.



Google
Cloud
Storage

Cloud
SQL

Cloud
Firestore

Google Cloud Storage (2010)

- Una solución para almacenar e interactuar con datos no estructurados, con una interfaz REST. La solución de object store de Google (competencia de S3)
- No es un filesystem tradicional, aunque la experiencia de uso es parecida. Pero los objetos son inmutables, no existen directorios, no tengo operaciones POSIX, etc
- Tiene muy alta durabilidad (11 nueves) lo que hace casi imposible perder un archivo. Esto lo logran con mecanismos de reconstrucción, replicación geográfica de los datos (al menos dos zonas, con posibilidad de multiregion) y validaciones constantes de los datos.
- Además al estar construido sobre **Colossus**, el sucesor de GFS, nos permite tener capacidad ""infinita"" y es administrada por **Spanner**, lo que garantiza consistencia fuerte para la metadata

```python
from google.cloud import storage


def upload_blob(bucket_name, source_file_name, destination_blob_name):
    """Uploads a file to the bucket."""
    # The ID of your GCS bucket
    # bucket_name = "your-bucket-name"
    # The path to your file to upload
    # source_file_name = "local/path/to/file"
    # The ID of your GCS object
    # destination_blob_name = "storage-object-name"

    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(destination_blob_name)

    # Optional: set a generation-match precondition to avoid potential race conditions
    # and data corruptions. The request to upload is aborted if the object's
    # generation number does not match your precondition. For a destination
    # object that does not yet exist, set the if_generation_match precondition to 0.
    # If the destination object already exists in your bucket, set instead a
    # generation-match precondition using its generation number.
    generation_match_precondition = 0

    blob.upload_from_filename(source_file_name, if_generation_match=generation_match_precondition)

    print(
        f"File {source_file_name} uploaded to {destination_blob_name}."
    )
```

```python
from google.cloud import storage


def download_blob(bucket_name, source_blob_name, destination_file_name):
    """Downloads a blob from the bucket."""
    # The ID of your GCS bucket
    # bucket_name = "your-bucket-name"

    # The ID of your GCS object
    # source_blob_name = "storage-object-name"

    # The path to which the file should be downloaded
    # destination_file_name = "local/path/to/file"

    storage_client = storage.Client()

    bucket = storage_client.bucket(bucket_name)

    # Construct a client side representation of a blob.
    # Note `Bucket.blob` differs from `Bucket.get_blob` as it doesn't retrieve
    # any content from Google Cloud Storage. As we don't need additional data,
    # using `Bucket.blob` is preferred here.
    blob = bucket.blob(source_blob_name)
    blob.download_to_filename(destination_file_name)

    print(
        "Downloaded storage object {} from bucket {} to local file {}.".format(
            source_blob_name, bucket_name, destination_file_name
        )
    )
```

```
gcloud storage cp gs://BUCKET_NAME/OBJECT_NAME SAVE_TO_LOCATION
```

```
gcloud storage cp OBJECT_LOCATION gs://DESTINATION_BUCKET_NAME
```

Bucket Locations

When you choose the location for a Cloud Storage bucket, consider the differences in availability, and price, as shown in the following table.

	Zone	Region	Dual-region	Multi-region
Availability	• Data redundancy within one availability zone • Zonal outages might affect availability	• Data redundancy across availability zones (synchronous) • RTO(recovery time objective)=0: automated failover and failback on zonal failure (no need to change storage paths)	• Higher availability than regions for a given storage class • Data redundancy across regions (asynchronous) • Turbo replication option for replication within 15 minutes • RTO(recovery time objective)=0: automated failover and failback on regional failure (no need to change storage paths)	• Higher availability than regions for a given storage class • Data redundancy across regions (asynchronous) • RTO(recovery time objective)=0: automated failover and failback on regional failure (no need to change storage paths)
Pricing	• Offers the highest performance in Cloud Storage for demanding workloads at the highest storage price, out of all location types • Zonal bucket-specific data transfer charges apply. For more information, see Cloud Storage pricing.	• Lowest storage price • No replication charges • No outbound data transfer charges when reading data inside the same region	• Highest storage price • Replication charges apply on write • No outbound data transfer charges when reading data within either region	• Higher storage price than regions, but lower than dual-regions • Replication charges apply on write • Outbound data transfer charges always apply when reading data

Location recommendations

Requirements	Recommended bucket location	Workload examples
• Ultra-low latency • Ultra-high bandwidth and I/O • High performance	Zone[1]	• Artificial Intelligence (AI) and Machine Learning (ML) • High-performance computing • Data-intensive analytics
• Optimized latency and bandwidth • Lowest data storage cost • Cross-zone redundancy	Region[2]	• Analytics • Backup and archive
• Optimized latency and bandwidth • Cross-region redundancy, with precise control of the locations where copies of data are stored	Dual-region[3]	• Analytics • Backup and archive • Disaster recovery
• Cross-geography data access • Cross-region redundancy	Multi-region	• Content serving

Clases de buckets

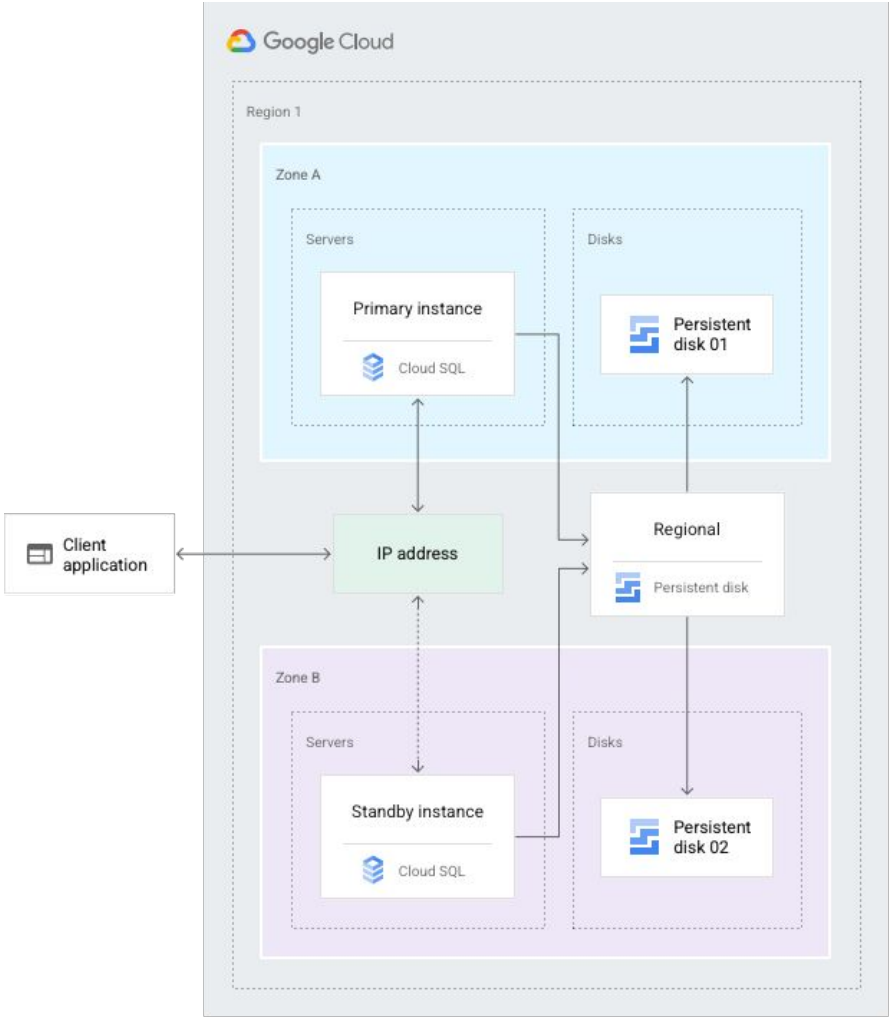
Rapid storage: diseñado para correr tareas con muchas operaciones de I/O. Tenemos que configurarlo en una zona específica de forma de poder minimizar los delays de red entre los procesos (AI/ML, data analysis) y los datos

Standard storage: ideal para lectura frecuente de un conjunto de archivos. Con replicación regional (dual o multi) se mejora la disponibilidad y se reduce la latencia de acceso si el recurso tiene que estar disponible en una zona amplia.

Nearline, Coldline y Archive: en este orden, el costo de almacenamiento es cada vez más barato pero el costo de lectura es más alto y la disponibilidad baja marginalmente. Además nos piden compromisos de por cuanto tienen que existir los datos (30, 90 y 365 días, respectivamente)

CloudSQL (2015)

- La oferta de *managed databases* de Google
- Actualizaciones, parches, scaling vertical de las instancias y auto-resizing del disco on-the-fly
- **High-availability o regional instances**: se crean dos instancias, una primaria y otra standby. Si falla la primaria la standby toma su lugar. Cuando la que era primary vuelve a responder se destruye y se replica a partir de la nueva primary, creando así la nueva failover.
- **Backups automatizados**, que nos permiten recuperarnos de un delete from equivocado. Corren y se administran de forma automática. Son incrementales y volver a un momento particular es muy fácil.
- **Read replicas** para descargar presión de lectura de la instancia primaria


Google Cloud
Region 1
Zone A
Servers
Primary instance
Cloud SQL
Disks
Persistent disk 01
Client application
IP address
Regional
Persistent disk
Zone B
Servers
Standby instance
Cloud SQL
Disks
Persistent disk 02

Firestore (2017)

- Una base de datos NoSQL basada en documentos
- Los documentos (JSON de hasta 1MB) se organizan en colecciones
- Tienen dos modos de operación: Native y Datastore. Native tiene un conjunto de features que nos permite usar Firestore como un Backend as a Service (BaaS). Tiene límites en la capacidad de escritura (10k escrituras/segundo)
- Datastore se suele usar cuando tenemos un backend que interactúa con nuestra instancia. No tiene disponibles limites ni features de BaaS
- Solo nos deja hacer consultas si tenemos índices para nuestra consulta.El índice de un solo campo se crea automáticamente, pero los índices compuestos deben definirse manualmente.

Firestore (2017)

- Es serverless, por lo que solo nos cobran por lecturas o escrituras. Hay que considerar que las escrituras consideran el mantenimiento de los índices
- Obviamente hay un costo por GiB
- Los datos se organizan en splits. Es una implementación de sharding pero fully managed por Firestore: nosotros solo tenemos que elegir la sharding key (ojo con las key secuenciales) y Firestore se encarga de administrar los splits por tamaño o por frecuencia de lectura, además de juntar los datos que se suelen consultar juntos en los mismos splits
- Esto nos permite escalar prácticamente de forma infinita sin una degradación de la performance